

UNIVERSITY OF MINES AND TECHNOLOGY
FACULTY OF COMPUTING AND MATHEMATICAL SCIENCES



CY376: Network Monitoring, Security and Auditing

END-OF-SEMESTER PROJECT REPORT

Auditing Stored Procedures and Database Objects for Security Weaknesses

Team Classification: Blue Team

Student Name: Veronica Okyere (FCM.41.018.206.23)

GitHub Repository: <https://github.com/cy-vokyere3623-hash/CY376-Blue-Team-Stored-Procedure-Audit>

Date: August 2026

Abstract

Stored procedures and related database objects are common components of enterprise applications, yet insecure procedure design and weak database configuration remain frequent sources of compromise. This Blue Team project investigated security weaknesses in stored procedures and database objects within an isolated Microsoft SQL Server 2022 Express laboratory using the AdventureWorks2022 sample database. The problem addressed was how defenders can systematically inventory database modules, detect dangerous coding and permission patterns, assess server surface area, and introduce detective controls before attackers abuse those weaknesses.

The methodology combined Centre for Internet Security (CIS) SQL Server guidance, OWASP SQL injection prevention principles, Microsoft SQL Server Audit capabilities, and NetSPI detective-control practices. Intentionally vulnerable lab procedures were created to simulate SQL injection through dynamic SQL, privilege escalation via EXECUTE AS OWNER, an xp_cmdshell wrapper, and excessive public execute grants. Custom T-SQL audit scripts then inventory modules, hunt dangerous code patterns, review permissions, check server configuration, and inspect audit status. Six findings were documented, remediated, and re-verified. After remediation, dangerous code-pattern queries returned zero rows, public execute grants on lab procedures were removed, SQL Server Audit named BlueTeamLabAudit was started, and remote access was disabled. The project demonstrates a complete blue-team workflow of inventory, detection, documentation, remediation, and verification for database object security.

Table of Contents

Abstract	2
1. Introduction.....	5
1.1 Background	5
1.2 Problem Statement	5
1.3 Aim and Objectives.....	5
1.4 Scope and Limitations.....	6
2. Literature and Tooling Review	7
2.1 CIS Microsoft SQL Server Benchmarks.....	7
2.2 OWASP SQL Injection Guidance	7
2.3 Microsoft SQL Server Audit and Catalog Views	7
2.4 NetSPI Detective Controls and Related Practice	8
2.5 Tools Used in This Project.....	8
3. Methodology	9
3.1 Laboratory Design	9
3.2 Audit Workflow	10
3.3 Finding Severity Model	11
4. Implementation	12
4.1 Creation of Vulnerable Laboratory Procedures	12
4.2 Inventory and Detection Scripts.....	12
4.3 Remediation Implementation.....	13
5. Results and Findings	15
5.1 Summary of Findings.....	15
5.2 Before Remediation Evidence	16
5.3 After Remediation Evidence	17

6. Analysis and Recommendations	20
6.1 Interpretation of Results.....	20
6.2 Recommendations	20
6.3 Practical Implications for Blue Teams.....	21
7. Conclusion	22
References	23
Appendix A. Project Folder Structure	24
Appendix B. Key Script Excerpts	24
Appendix C. Evidence Screenshot Index.....	24

1. Introduction

1.1 Background

Modern applications frequently rely on stored procedures to encapsulate business logic close to data. From a defensive perspective, this design can improve consistency and reduce direct table access, but it does not automatically guarantee safety. Stored procedures may still construct dynamic SQL insecurely, execute under elevated ownership contexts, call high-risk extended procedures, or inherit overly broad permissions through the public role. When such weaknesses exist, attackers who reach the database layer can escalate privileges, extract sensitive information, or move toward operating-system command execution.

Blue team work focuses on reducing exposure and improving detection before incidents occur. In database environments, that includes inventorying modules, reviewing definitions and permissions, hardening configuration against CIS benchmarks, and enabling audit trails that record sensitive actions. This project therefore treats stored procedure and database object auditing as a practical defensive control activity rather than a purely theoretical exercise.

1.2 Problem Statement

Many student and production labs emphasise application-layer testing while giving less structured attention to database object hygiene. Without a repeatable audit method, dangerous patterns such as string-concatenated EXEC statements, EXECUTE AS OWNER, xp_cmdshell wrappers, and public execute grants can remain unnoticed. The problem addressed by this project is how a blue team analyst can establish a lab, create representative weaknesses, detect them with evidence, remediate them, and prove the fix.

1.3 Aim and Objectives

The aim of the project was to audit stored procedures and related database objects for security weaknesses in a controlled SQL Server laboratory and to remediate the identified issues.

Specific objectives were to:

1. Set up SQL Server 2022 Express, SSMS, and AdventureWorks2022 as an isolated lab.

2. Create intentionally vulnerable stored procedures representing common weaknesses.
3. Develop and run T-SQL scripts for inventory, code-pattern hunting, permission review, configuration checks, and audit inspection.
4. Document findings with severity ratings and supporting screenshots.
5. Remediate the findings and re-verify with the same audit scripts.
6. Produce a report, evidence package, and GitHub-ready project structure suitable for CY376 submission.

1.4 Scope and Limitations

The scope was limited to a local SQL Server Express instance (localhost\SQLEXPRESS) and the AdventureWorks2022 database. The project did not attack production systems and did not include network intrusion detection, SIEM correlation beyond Windows Application log concepts, or full CIS-CAT automated scoring. Linked-server abuse and SQL Agent job abuse were reviewed conceptually through literature and NetSPI guidance but were not the primary lab targets. All vulnerable procedures were lab-only artefacts created for training and then remediated.

2. Literature and Tooling Review

2.1 CIS Microsoft SQL Server Benchmarks

The Centre for Internet Security publishes consensus benchmarks for Microsoft SQL Server that define secure configuration baselines (Center for Internet Security, n.d.). Relevant controls for this project include surface-area reduction for features such as xp_cmdshell, Ole Automation Procedures, CLR, and Ad Hoc Distributed Queries; least privilege for roles including public; and auditing and logging expectations. The benchmark approach shaped the server-configuration script used in Phase 5 of the lab and provided an industry-recognised reference for classifying configuration findings.

2.2 OWASP SQL Injection Guidance

The OWASP SQL Injection Prevention Cheat Sheet emphasises that stored procedures are not inherently safe (OWASP Foundation, n.d.). Procedures become unsafe when developers concatenate untrusted input into dynamic SQL and execute it with EXEC or similar constructs. OWASP therefore recommends parameterized queries and careful review of sp_executesql, EXEC, and related patterns inside procedure definitions. This guidance directly informed finding F-001 and the design of script 02-code-patterns.sql.

2.3 Microsoft SQL Server Audit and Catalog Views

Microsoft documentation describes sys.sql_modules as the catalog view that returns definitions for SQL modules such as procedures, functions, triggers, and views (Microsoft, n.d.-a). Combined with sys.objects, sys.schemas, and sys.database_permissions, these views enable defensive inventories without requiring proprietary scanners. Microsoft SQL Server Audit further provides server and database audit specifications that can log sensitive actions to the Windows Application log (Microsoft, n.d.-b). Enabling audit for configuration changes and selected procedure execution formed the remediation for finding F-005.

2.4 NetSPI Detective Controls and Related Practice

NetSPI's SQL Server Detective Control Cheat Sheet documents practical audit specifications for detecting dangerous activity such as xp_cmdshell execution, OLE automation, external scripts, and linked-server abuse (Sutherland, n.d.). Although this lab focused on module and permission weaknesses, NetSPI's work justified creating a named server audit (BlueTeamLabAudit) and associating server and database specifications rather than relying only on ad hoc queries.

2.5 Tools Used in This Project

The following tools supported implementation and evidence collection:

- Microsoft SQL Server 2022 Express Edition as the database engine.
- SQL Server Management Studio (SSMS) 21 for connection, script execution, and screenshots.
- AdventureWorks2022 sample database as a realistic schema for lab procedures.
- Custom T-SQL scripts for inventory, pattern hunting, permissions, configuration, and audit checks.
- Windows PowerShell and sqlcmd for automation and verification.
- GitHub for repository submission of scripts, evidence, and the final report.

3. Methodology

3.1 Laboratory Design

The laboratory was hosted on a Windows workstation named NOVA and used Windows Authentication as NOVA\Administrator. SQL Server Express listened as the named instance localhost\SQLEXPRESS. The AdventureWorks2022 database provided tables such as Production.Product and HumanResources.Employee for realistic procedure targets. Figure 1 summarises the topology.

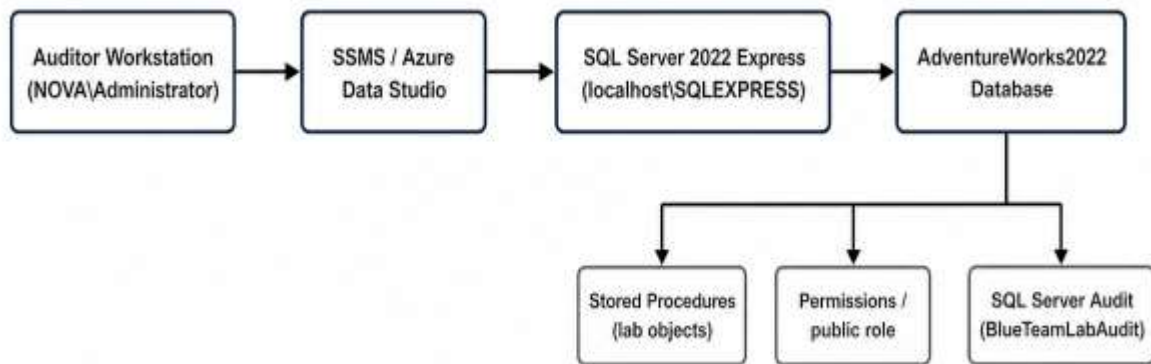


Figure 1. Blue Team laboratory topology for the SQL Server stored procedure audit.

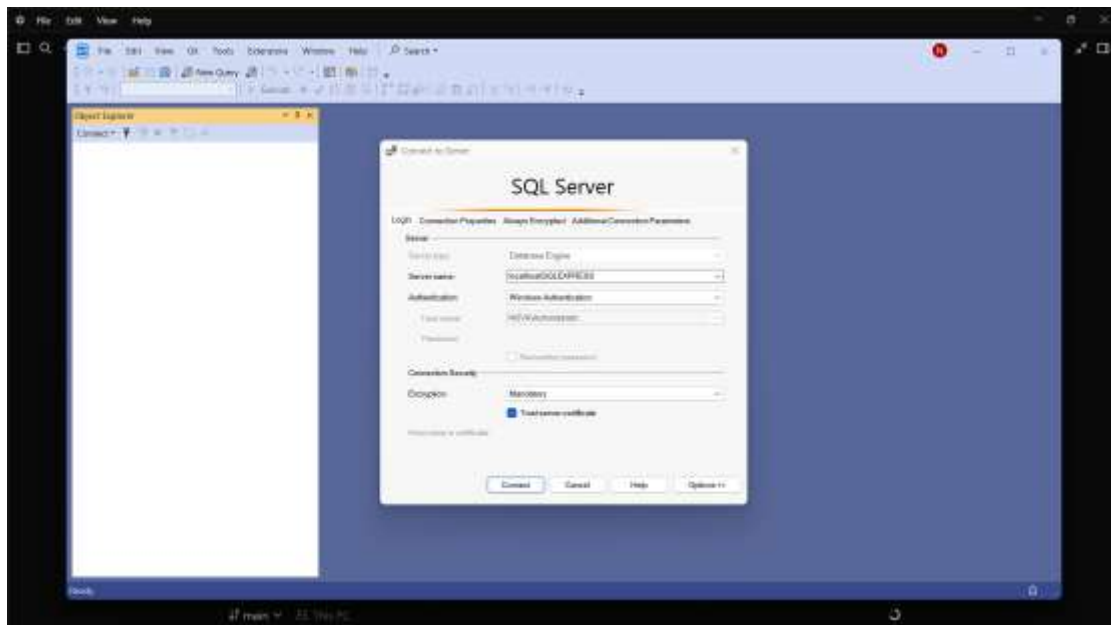


Figure 2. SSMS connection dialog for localhost\SQLEXPRESS using Windows Authentication.

3.2 Audit Workflow

The workflow followed five defensive stages: (1) prepare the lab and create representative vulnerable objects; (2) inventory modules; (3) hunt dangerous code and permission patterns; (4) document findings; and (5) remediate and re-verify. This sequence mirrors standard blue-team practice in which detection is incomplete unless findings are fixed and tested again.

Table 1 maps project phases to scripts and purpose.

Phase	Script / Artefact	Purpose
1 Setup	setup-vulnerable-procs.sql	Create intentionally weak lab procedures
2 Inventory	01-inventory.sql	List modules, EXECUTE AS mode, visibility
3 Code hunt	02-code-patterns.sql	Find EXEC/xp_cmdshell/EXECUTE AS risks
4 Permissions	03-permissions.sql	Review public and lab execute grants
5 Config	04-server-config.sql	CIS surface-area and sysadmin review
6 Audit check	05-audit-check.sql	List SQL Server Audit status/specs
7 Remediate	remediate-findings.sql	Fix findings and enable audit
8 Verify	Re-run 02–05	Confirm risks closed with evidence

Table 1. Mapping of blue-team phases to laboratory artefacts.

3.3 Finding Severity Model

Findings were rated Critical, High, Medium, or Low. Critical covered clear SQL injection and OS command paths. High covered privilege escalation and excessive public grants. Medium covered missing detective controls. Low covered residual surface-area issues such as remote access when other dangerous features were already disabled.

4. Implementation

4.1 Creation of Vulnerable Laboratory Procedures

Three intentionally weak procedures were deployed into AdventureWorks2022 to create measurable audit targets. `usp_LabSearchProducts_Unsafe` concatenated `@ProductName` into dynamic SQL and executed it with `EXEC(@sql)`, illustrating OWASP's warning about unsafe stored procedures. `usp_LabGetEmployee_Elevate` used `WITH EXECUTE AS OWNER` to demonstrate privilege escalation risk. `usp_LabRunCommand_Dangerous` wrapped `master.dbo.xp_cmdshell`. `EXECUTE` permission on all three was granted to public. Figure 3 shows successful deployment of the vulnerable objects.

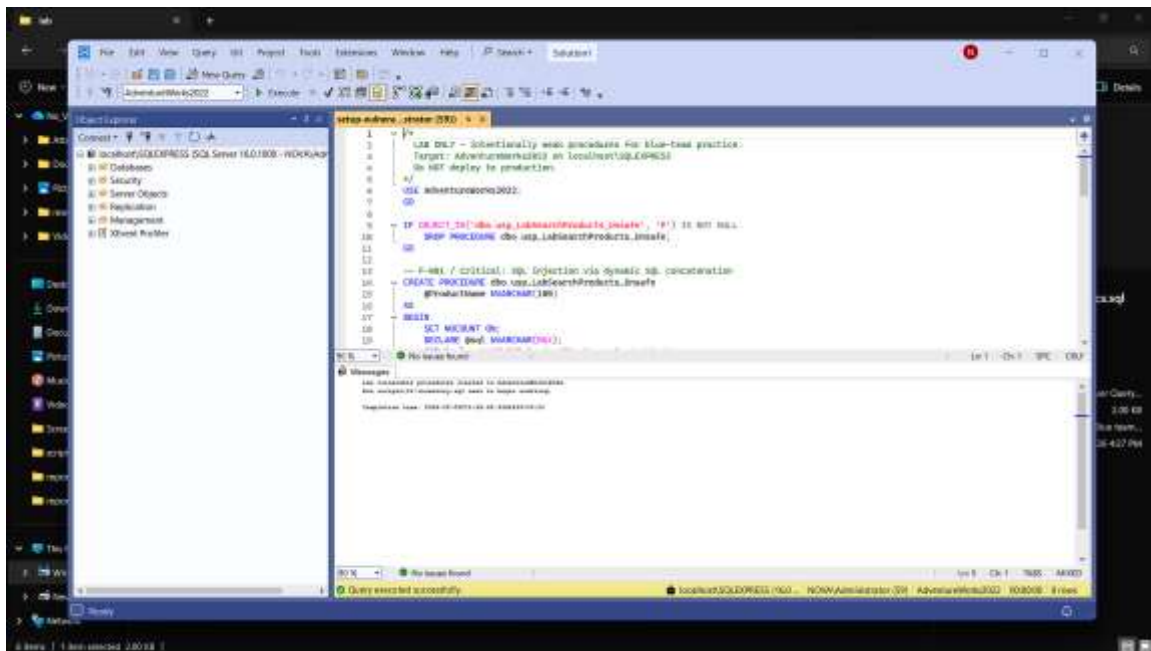


Figure 3. Successful creation of intentionally vulnerable lab stored procedures.

4.2 Inventory and Detection Scripts

Script 01-inventory.sql joined `sys.sql_modules`, `sys.objects`, and `sys.schemas` to list module names, types, timestamps, `EXECUTE AS` mode, and whether definitions were visible or encrypted. Script 02-code-patterns.sql searched module definitions for `EXEC()`, `sp_executesql`, `xp_cmdshell`, `OLE`, `OPENROWSET`, and related tokens, and separately listed procedures with non-null `execute_as_principal_id`. Script 03-permissions.sql reviewed public grants and focused

EXECUTE grants on usp_Lab% objects. Script 04-server-config.sql queried sys.configurations for CIS-relevant options and listed sysadmin members. Script 05-audit-check.sql inspected dm_server_audit_status and audit specification details.

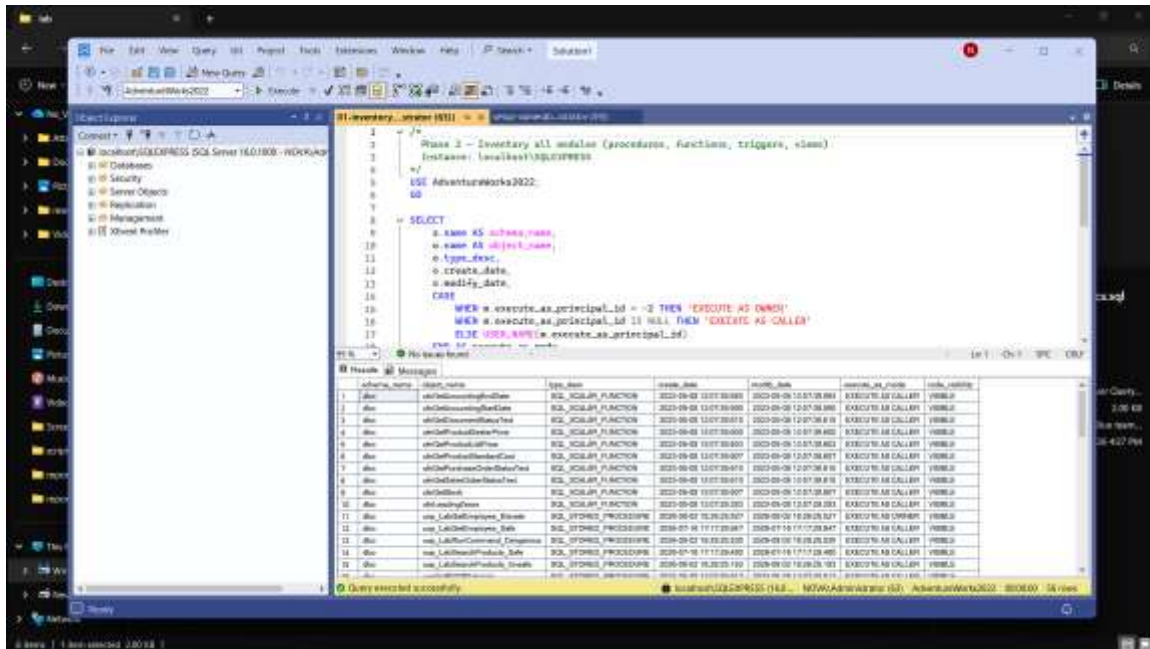


Figure 4. Inventory results highlighting lab procedures and EXECUTE AS OWNER.

4.3 Remediation Implementation

Remediation was implemented in lab\remediate-findings.sql. Unsafe and elevated procedures were dropped. Safer replacements usp_LabSearchProducts_Safe and usp_LabGetEmployee_Safe used static parameterized-style logic without EXECUTE AS OWNER. The xp_cmdshell wrapper was removed without replacement. Public execute grants on lab procedures were revoked. Remote access was set to 0. A server audit BlueTeamLabAudit was created to the Application log with a server audit specification for AUDIT_CHANGE_GROUP and SERVER_OPERATION_GROUP, plus a database audit specification monitoring execute on the safe lab procedures.

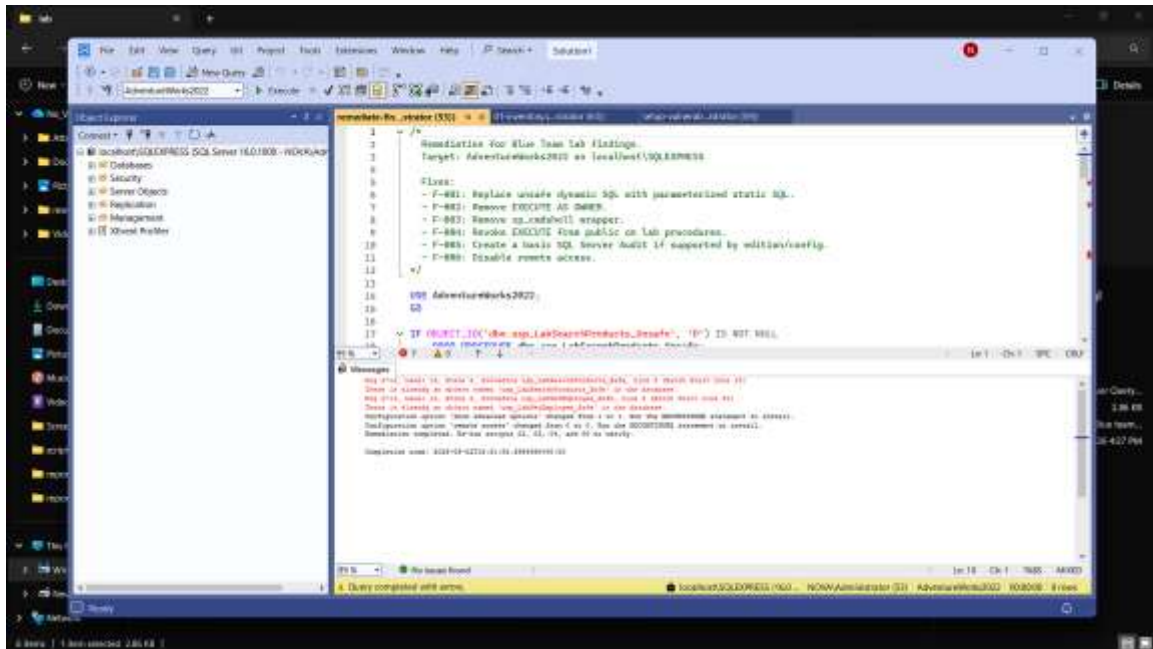


Figure 5. Execution of the remediation script against AdventureWorks2022.

5. Results and Findings

5.1 Summary of Findings

The audit produced six findings. Table 2 summarises identifier, severity, status, and object. All findings were remediated and verified before final reporting.

ID	Severity	Status	Object / Area	Issue
F-001	Critical	Fixed	usp_LabSearchProducts_Unsafe	Dynamic SQL injection pattern
F-002	High	Fixed	usp_LabGetEmployee_Elevate	EXECUTE AS OWNER elevation
F-003	Critical	Fixed	usp_LabRunCommand_Dangerous	xp_cmdshell wrapper
F-004	High	Fixed	usp_Lab* / public	Public EXECUTE grants
F-005	Medium	Fixed	SQL Server Audit	No audit configured initially
F-006	Low	Fixed	remote access	Remote access enabled

Table 2. Consolidated findings from the stored procedure and database object audit.

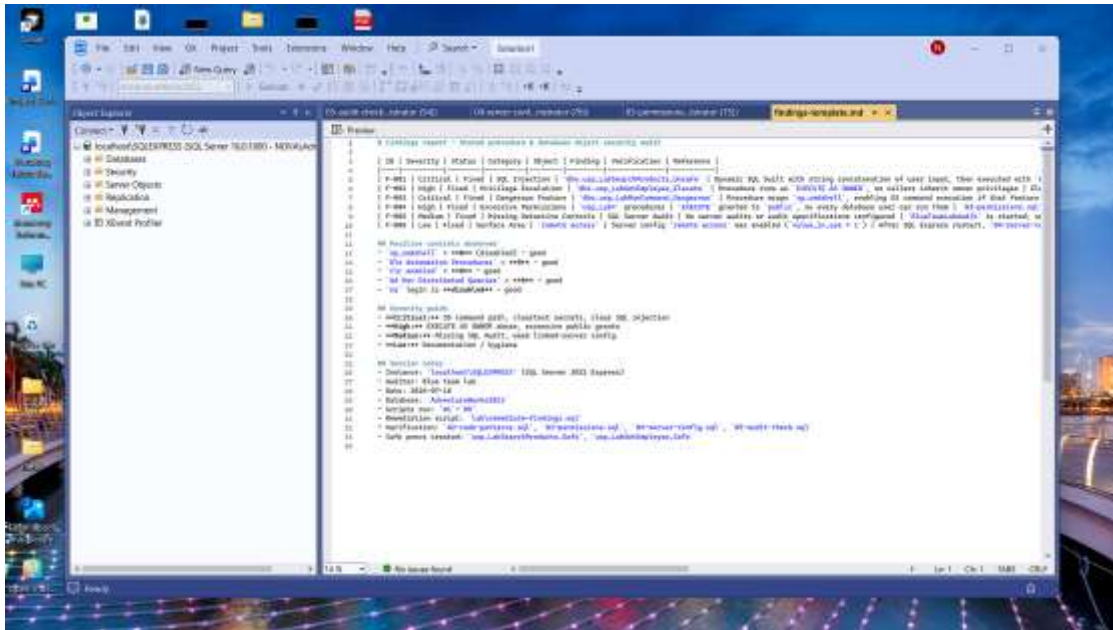


Figure 6. Documented findings register with severity, status, and verification notes.

5.2 Before Remediation Evidence

Before remediation, code-pattern hunting returned `usp_LabRunCommand_Dangerous` and `usp_LabSearchProducts_Unsafe`, while the elevation query returned `usp_LabGetEmployee_Elevate` with EXECUTE AS OWNER. Permission review showed public EXECUTE GRANT on all three lab procedures. These results confirmed that the intentionally weak objects were detectable with the audit scripts.

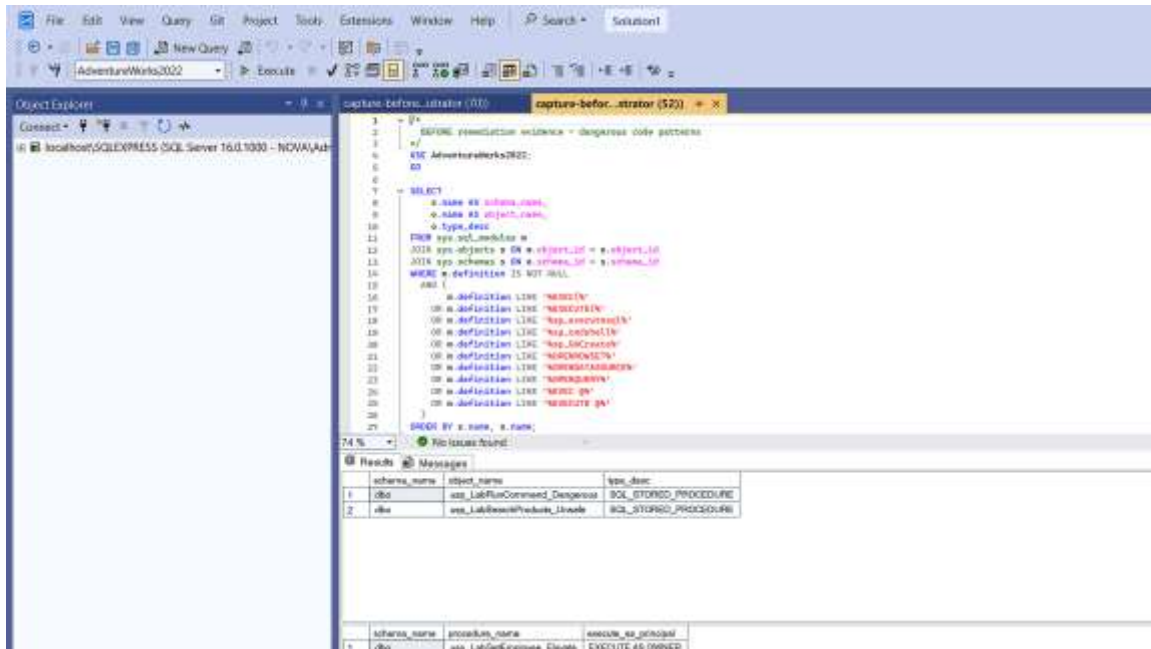


Figure 7. SSMS results before remediation: dangerous code patterns and EXECUTE AS OWNER.

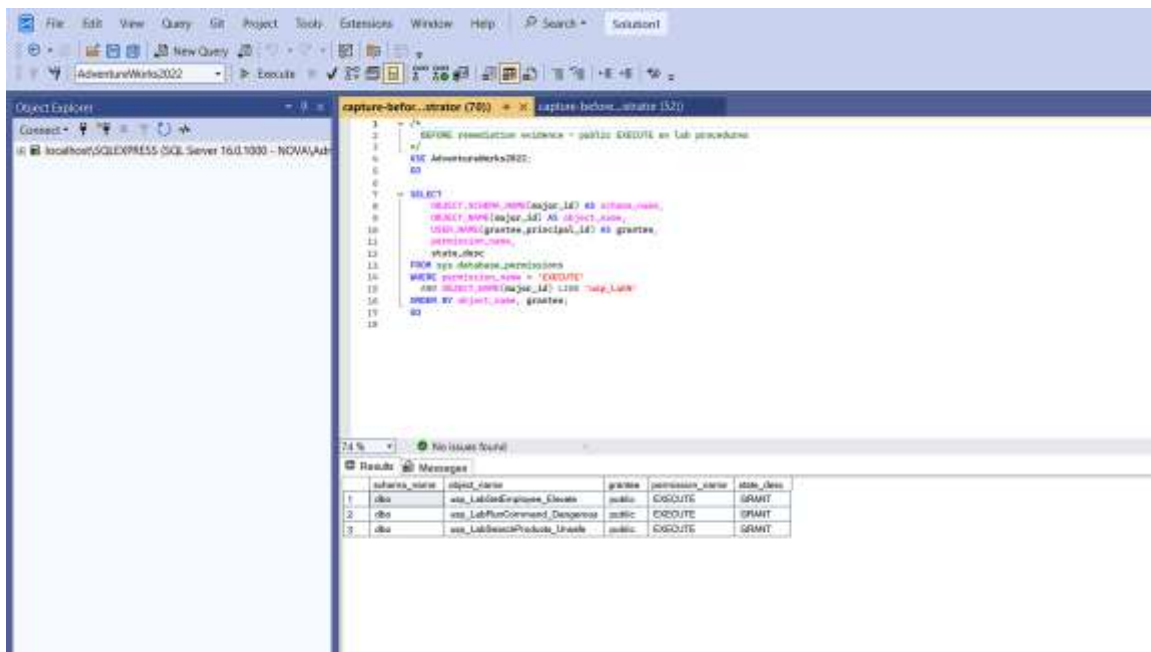


Figure 8. SSMS results before remediation: public EXECUTE grants on lab procedures.

5.3 After Remediation Evidence

After remediation, script 02-code-patterns.sql returned zero rows for both dangerous definitions and elevated execute-as procedures. Server configuration checks showed

The screenshot displays the Microsoft SQL Server Enterprise interface. The main window shows a query plan for a SELECT statement. The query is: `SELECT * FROM sys.dm_exec_requests WHERE session_id = 50 AND status = 'WAITING'`. The query plan indicates a table scan of `sys.dm_exec_requests` with a filter on `session_id` and `status`. The estimated number of rows is 1. The status bar at the bottom indicates 'Query executed successfully'.

[illegible]

Figure 10. CIS surface-area configuration results after hardening.

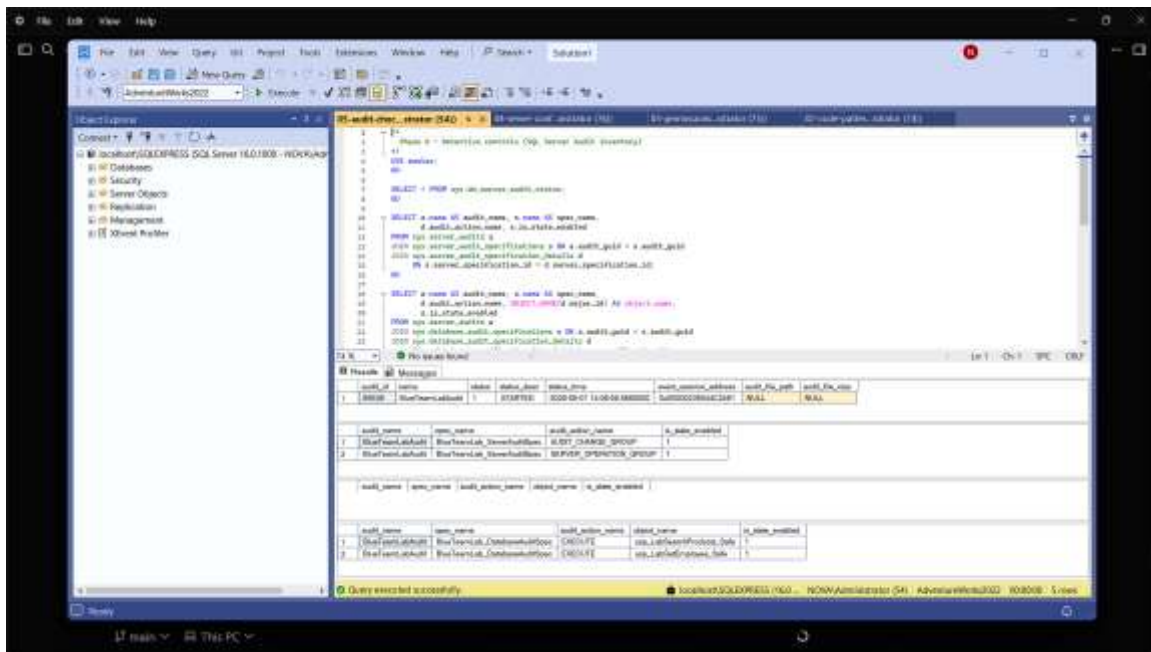


Figure 11. BlueTeamLabAudit started with server and database audit specifications.

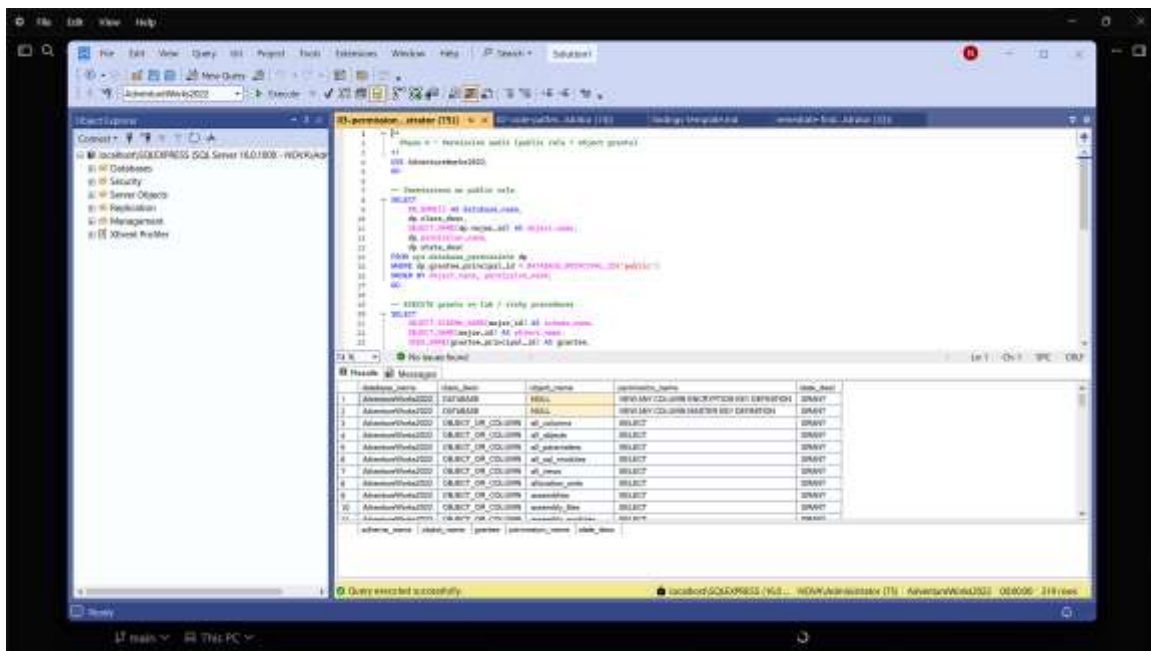


Figure 12. Permission audit script execution used during review and verification.

6. Analysis and Recommendations

6.1 Interpretation of Results

The before-and-after evidence demonstrates that common database weaknesses are both detectable and fixable when defenders use catalog views and a structured checklist. F-001 and F-003 were critical because they map to direct data compromise and potential host command execution paths. Even though `xp_cmdshell` was disabled at the server level during much of the lab, retaining a wrapper procedure and public execute rights would become immediately dangerous if the feature were re-enabled. F-002 showed that ownership chaining through `EXECUTE AS OWNER` can silently expand caller privilege. F-004 confirmed that public grants convert object-level bugs into broadly reachable attack surfaces. F-005 showed that without SQL Server Audit, configuration and procedure abuse may leave insufficient forensic trail. F-006 was lower severity in this Express lab but still aligned with CIS surface-area reduction.

6.2 Recommendations

Based on the findings, the following recommendations are made for similar environments:

7. Prohibit dynamic SQL built by string concatenation; require parameterized `sp_executesql` or static SQL where possible.
8. Avoid `EXECUTE AS OWNER` unless a documented least-privilege impersonation account is required and reviewed.
9. Do not grant `EXECUTE` on application procedures to public; use dedicated application roles.
10. Keep `xp_cmdshell`, Ole Automation, and related high-risk features disabled unless an exception is approved.
11. Enable SQL Server Audit (or equivalent) for audit-change, server-operation, and sensitive procedure execution events.
12. Re-run inventory and code-pattern scripts after every significant schema deployment as a regression check.
13. Store audit scripts and findings under version control so blue-team work remains repeatable and reviewable.

6.3 Practical Implications for Blue Teams

For blue teams, the main lesson is that database security monitoring should include object definitions and grants, not only failed logins. Catalog queries are low-cost controls that can be scheduled or run during change windows. Combining preventive hardening with detective audit specifications creates defence in depth: unsafe objects are harder to introduce, and attempts to alter audit posture or execute sensitive procedures are more likely to leave evidence.

7. Conclusion

This project completed a Blue Team audit of stored procedures and database objects on SQL Server 2022 Express using AdventureWorks2022. The work established a controlled laboratory, introduced representative weaknesses, and applied a multi-script audit method grounded in CIS, OWASP, Microsoft, and NetSPI guidance. Six findings spanning SQL injection patterns, privilege escalation, dangerous feature wrappers, excessive permissions, missing audit, and remote access were documented with SSMS evidence.

Remediation removed unsafe objects, introduced safer replacements where appropriate, revoked public execute grants, disabled remote access, and started BlueTeamLabAudit. Re-verification showed zero dangerous code-pattern rows and confirmed hardened configuration and active audit specifications. The outcome is a demonstrable defensive lifecycle: inventory, detect, document, remediate, and verify.

Future work could extend the same method to linked servers, SQL Agent jobs, encrypted modules, and automated CIS benchmark scoring, and could forward Windows Application log event 33205 into a SIEM for continuous monitoring. Within the CY376 scope, however, the project objectives were met and no open findings remain in the laboratory.

References

Center for Internet Security. (n.d.). CIS Microsoft SQL Server benchmarks.

https://www.cisecurity.org/benchmark/microsoft_sql_server

Microsoft. (n.d.-a). sys.sql_modules (Transact-SQL). Microsoft Learn.

<https://learn.microsoft.com/en-us/sql/relational-databases/system-catalog-views/sys-sql-modules-transact-sql>

Microsoft. (n.d.-b). SQL Server audit action groups and actions. Microsoft Learn.

<https://learn.microsoft.com/en-us/sql/relational-databases/security/auditing/sql-server-audit-action-groups-and-actions>

Microsoft. (n.d.-c). AdventureWorks sample databases. Microsoft Learn.

<https://learn.microsoft.com/en-us/sql/samples/adventureworks-install-configure>

OWASP Foundation. (n.d.). SQL injection prevention cheat sheet.

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

Sutherland, S. (n.d.). SQL Server detective control cheat sheet. NetSPI PowerUpSQL Wiki.

<https://github.com/NetSPI/PowerUpSQL/wiki/SQL-Server-Detective-Control-Cheat-Sheet>

Appendix A. Project Folder Structure

The GitHub-oriented project layout used for submission is summarised below.

```
Blue team Broni/  
  README.md  
  scripts/01-inventory.sql ... 05-audit-check.sql  
  lab/setup-vulnerable-procs.sql  
  lab/remediate-findings.sql  
  docs/findings-template.md  
  evidence/screenshots/  
  reports/CY376-Blue-Team-Report.docx
```

Appendix B. Key Script Excerpts

The following excerpt from the vulnerable search procedure illustrates the unsafe concatenation pattern identified as F-001:

```
SET @sql = N'SELECT ProductID, Name, ProductNumber  
            FROM Production.Product  
            WHERE Name LIKE ''%' + @ProductName + N'%' ''';  
EXEC(@sql); -- UNSAFE
```

The remediation replaced unsafe objects with safer procedures and enabled BlueTeamLabAudit. Full scripts are included in the repository under scripts\ and lab\ rather than duplicated in full here to keep the main body readable, as required by the submission guidelines for appendices versus core narrative.

Appendix C. Evidence Screenshot Index

Figure	File	Description
1	10-lab-topology-diagram.png	Lab topology
2	01-ssms-connect-...	SSMS connection
3	02-setup-vulnerable-...	Vulnerable procs created

4	03-inventory-...	Inventory / EXECUTE AS OWNER
5	04-remediate-...	Remediation run
6	09-findings-template-...	Findings register
7	11-code-patterns-before-...	Before: code patterns
8	12-permissions-...-before	Before: public EXECUTE
9	05-code-patterns-after-...	After: 0 risky rows
10	07-server-config-...	CIS surface area
11	08-sql-audit-...	Audit started
12	06-permissions-...	Permissions audit view